Range-Vectorized Discounted Regret Minimization for No-Limit Hold'em: Complexity, Precision and Parallelism

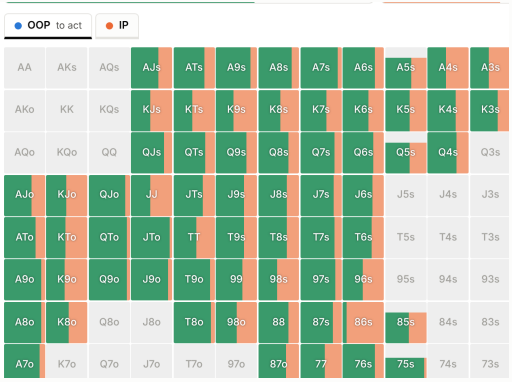
Numerical Analysis Research Club

Background and scope

Programs based on counterfactual regret minimization (CFR) defeated professional players at heads-up no-limit hold'em in 2017 and at six-player hold'em in 2019.

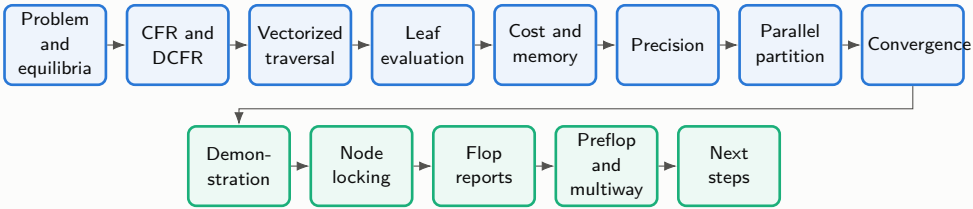
Commercial solvers compute equilibria of user-specified subgames: a board, two ranges and a set of bet sizes.

This work builds such a solver on one algorithm, discounted CFR, for the flop, turn and river, with node locking, aggregate reports and a preflop solver for two to six players, and analyses its numerics.



The big blind's turn strategy on K♠7♢2♣5♠ in the browser solver. Green is check and red is bet.

Outline



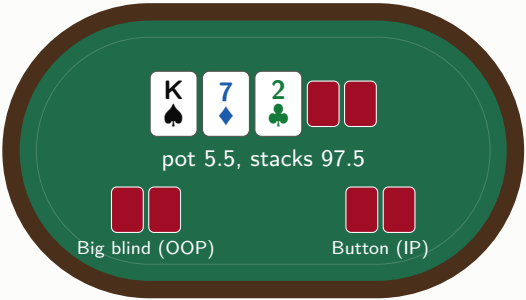
The first row is the solver core and its analysis. The second row shows the core at work and the extensions built on it.

The postflop subgame

Each player holds two private cards. The board has three, four or five public cards, and the remaining cards are dealt by chance between betting rounds.

A subgame is given by the board, the pot, the effective stack, a **range** for each player and a **bet tree** of allowed sizes. A range is a weight vector over the hands that avoid the board: at most 1176 on the flop, 1128 on the turn and 1081 on the river.

The unknowns are an action distribution for every hand at every decision node.



street	decision nodes	hands
river	17	449 / 428
turn	1,349	460 / 436
flop	56,842	488 / 460

Zero-sum games, equilibrium and exploitability

Behavioural strategies of the two players determine realization plans $x \in X$ and $y \in Y$, polytopes defined by flow constraints, and the expected payoff is bilinear.

Minimax theorem (von Neumann 1928)

$$\max_{x \in X} \min_{y \in Y} x^\top A y \;=\; \min_{y \in Y} \max_{x \in X} x^\top A y \;=:\; v^*.$$

A pair attaining both sides is a Nash equilibrium. Solution quality is measured by the **exploitability**

$$\mathrm{expl}(x, y) \;=\; \tfrac{1}{2}\Big(\max_{x'} x'^\top A y \;-\; \min_{y'} x^\top A y' \Big) \;\geq 0,$$

reported as a percentage of the pot. With rake the game is not zero-sum and the convergence theorems below do not apply. The solver then reports the mean gain of the two best responses.

Regret matching and counterfactual regret minimization

At every information set I, CFR keeps a cumulative regret per action and plays in proportion to its positive part:

$$R_T(I,a) = \sum_{t=1}^{T} \big(v_t(I,a) - v_t(I)\big), \qquad \sigma_{T+1}(I,a) = \frac{[R_T(I,a)]^+}{\sum_b [R_T(I,b)]^+}.$$

The values v_t are **counterfactual**: they are weighted by the probability that chance and the opponent reach I.

Theorem (Zinkevich, Johanson, Bowling and Piccione 2007)

The overall regret is bounded by the sum of the counterfactual regrets, so the average strategies satisfy

$$\mathrm{expl}(\bar{x}_T, \bar{y}_T) \;\le\; \frac{\Delta\,|\mathcal{I}|\sqrt{|A|}}{\sqrt{T}}.$$

The bound guarantees convergence; at practical T it is vacuous.

Discounted CFR

DCFR discounts the cumulative quantities at iteration t before adding the new ones:

$$R \leftarrow \begin{cases} R\,\dfrac{t^\alpha}{t^\alpha+1} & R > 0 \\ R\,\dfrac{t^\beta}{t^\beta+1} & R \le 0 \end{cases} + \big(v(a) - v\big), \qquad S \leftarrow S\Big(\frac{t}{t+1}\Big)^\gamma + \pi_{\text{own}}\,\sigma_t.$$

We use $(\alpha, \beta, \gamma) = (1.5, 0, 2)$ and update the players alternately. CFR is the limit $\alpha = \beta = \infty$, $\gamma = 0$, and CFR$^+$ is $\alpha = \infty$, $\beta = -\infty$, $\gamma = 1$.

Theorem (Brown and Sandholm 2019)

With $(\alpha, \beta, \gamma) = (1.5, 0, 2)$ and simultaneous updates, the weighted average strategies form a $6\Delta|\mathcal{I}|\big(\sqrt{|A|} + 1/\sqrt{T}\big)/\sqrt{T}$ equilibrium after T iterations.

The implementation computes exactly these iterates: subtrees that the opponent cannot reach are still visited to apply the discounts and strategy sums, and only their leaves are skipped.

Range-vectorized traversal of the public tree

The **public tree** has one node per betting sequence and dealt card. At a public node the information sets of the acting player are indexed by its hands, so every quantity is a vector of length n_p.

A traversal for player p carries the opponent's reach $r_{-p} \in \mathbb{R}^{n_{-p}}$ down and returns the values $v_p \in \mathbb{R}^{n_p}$ up. A decision node with $|A|$ actions costs $O(n|A|)$ arithmetic on contiguous memory.

Regrets and strategy sums are stored action-major, so the inner loops run over hands.

One traversal for player p

At an opponent node, recurse on each action a with reach $r_{-p} \odot \sigma(a)$ and add the child values.

At an own node, recurse on every action, set $v = \sum_a \sigma(a) \odot v_a$ and update R and S.

At a chance node, average the children over the dealt cards.

At a leaf, return the payoff against r_{-p}.

Chance nodes

Once both hands are fixed, $K = 48 - |\mathrm{board}|$ cards remain, and each of them is equally likely.

Lemma

For hands h_1, h_2 and a card c not in $h_1 \cup h_2 \cup \mathrm{board}$, $\Pr(c \mid h_1, h_2, \mathrm{board}) = 1/K$.

Hence the chance node returns

$$v_p(h) \;=\; \frac{1}{K} \sum_{c \notin \mathrm{board}} \mathbf{1}[c \notin h]\; v_p^{(c)}(h),$$

where the child for card c is evaluated with the opponent's hands containing c set to zero reach. The weight is the same constant for every hand pair, so chance needs no normalization by range.

Fold leaves in linear time

At a fold leaf the value of hand h is the payoff times the opponent weight that shares no card with h:

$$m(h) \;=\; \sum_{j\,:\,j\cap h=\emptyset} r_j \;=\; \tau - s_{h_1} - s_{h_2} + r_{\bar{h}}, \qquad \tau = \sum_j r_j, \quad s_c = \sum_{j\ni c} r_j.$$

The last term restores the opponent hand $\bar{h}$ equal to h, which was subtracted twice. The 52 card sums cost $O(n)$, and so does the evaluation for all h; the direct sum costs $O(n^2)$.

In matrix form the compatibility matrix is

$$\Gamma = \mathbf{1}\mathbf{1}^\top - \sum_{c=1}^{52} e_c f_c^\top + P,$$

where e_c and f_c indicate the hands containing card c and P matches identical hands: rank at most 53 plus a partial permutation.

Showdown leaves in linear time

Sort both lists by hand strength once per runout. One ascending sweep gives, for each own hand, the compatible opponent weight that is strictly weaker, $L(h)$, and weaker or equal, $L^{=}(h)$, with the same inclusion–exclusion inside the sweep. Then

$$v(h) = \ell\, m(h) + (w-\ell)\, L(h) + (\theta-\ell)\big(L^{=}(h) - L(h)\big),$$

where w, ℓ and θ are the payoffs for a win, a loss and a tie.

The sort is cached per runout, so each evaluation is $O(n)$.

n	fold	showdown	direct
50	0.5	1.2	10
100	0.5	2.2	46
200	0.9	3.0	276
400	2.3	5.5	1193
700	3.3	9.2	3773
1081	5.9	13.7	4272

Microseconds per evaluation for n hands. The direct method sums over all compatible pairs; the largest relative difference is 7×10^{-13}.

A cost model for one iteration

The time of one iteration is modelled by two work counts,

$$T \approx a\, W_{\mathrm{dec}} + b\, W_{\mathrm{show}}, \qquad W_{\mathrm{dec}} = \sum_{\text{decisions}} n_p |A|,$$

where W_{show} counts the hands at the showdown leaves evaluated in that iteration; fold and chance work are absorbed into a and b.

Times are median milliseconds per iteration on one core. A fit to 252 timed iterations of 8 trees gives $a = 30$ and $b = 17$ ns. Predicted from the other trees, the median error is 7 and 10% on the two largest flop trees and at most 30%. The model under-predicts leaf-dominated trees, so it misses some per-leaf overhead.

tree	decisions	measured	model	held out, %
turn, 1 size	580	31	23	30
turn, raises and all-in	2,216	95	101	13
turn, river checked down	10	7	7	23
turn, two sizes	2,504	121	118	12
flop, checked down later	8	316	259	21
flop, river checked down	789	506	456	12
flop, 1 size per street	56,842	2276	2277	7
flop, standard	85,263	3094	3395	10

Memory

Each decision node stores regrets and strategy sums for every hand and action. In single precision this is

$$\text{memory} \;=\; 8 \sum_{\text{decisions}} n_p \,|A| \text{ bytes.}$$

Storage is a few flat blocks of 2^{24} entries, and each node holds two views into them. This cut the allocation time of a 2.9 GB tree from 32 s to under a second.

The interface estimates the memory before building, from the top of the tree and one card's subtree; for the three trees we could build in full it is within 0.1%. The larger estimates are not verified.

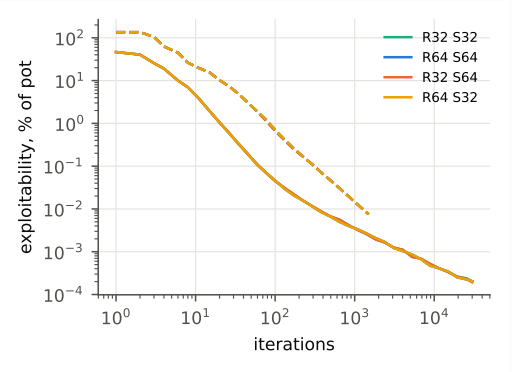
flop	turn	river	MB	built
– / 33	75 / 75	75 / 75	431	431
cr 50 / 33	75 / 75	75 / 75	647	647
cr 50 / 33	– / 75	75 / 75, ai	755	755
cr 50 / 33	66 / 66, r 50	75 / 75, ai	1,457	–
cr 50 / 33, 75	– / 75, r 50	75 / 75, ai	1,680	–
cr 60 / 33	75 / 75, r 60	75 / 75, r 60, ai	3,614	–

Flop K72; MB estimated and, where built, actual. Bet sizes in percent of the pot, out of position / in position; "cr 50" is a check-raise of 50%, "r 50" a raise of 50% and "ai" adds all-in. A dash means no bet.

Floating-point precision

Regrets R and strategy sums S are stored in single precision; all arithmetic inside a traversal is double precision.

All four combinations of single and double precision for R and S reach the same exploitability, down to 2×10^{-4}% of the pot. The strategies differ by a reach-weighted L^1 distance of 6.8×10^{-3} on the small game and 8.1×10^{-3} on the turn spot. Double precision runs whose ranges differ by 10^{-7} differ just as much (0.012 and 7.5×10^{-3}), so this is the sensitivity of the DCFR trajectory, not storage precision.



Exploitability with float32 or float64 storage of R and S, on a small turn game (solid) and the turn spot (dashed).

Parallel partition by the first chance card

Below the first chance node the subtrees for different cards are independent. Cards are dealt round-robin to P workers and a coordinator keeps the tree above. One traversal is three steps:

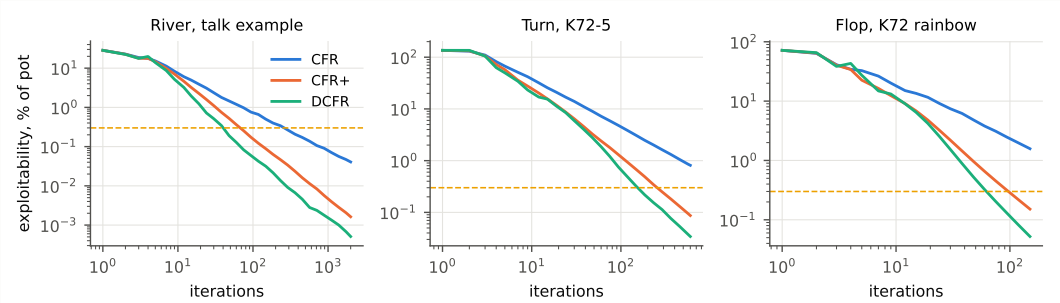
1. the coordinator records both reach vectors at each first-level chance node;
2. each worker returns, per chance node, the sum of its cards' child values;
3. the coordinator divides by K and finishes the traversal.

With t_c the coordinator's time and t_1 the time of one card, the model is $T(P) \approx t_c + \lceil K/P \rceil \, t_1$.

spot	seq.	$P = 1$	$P = 2$	model	speedup
turn	62	65	35	33	1.8
flop	2317	2327	1212	1187	1.9

Medians of seven interleaved runs in Chrome, ms per iteration; interquartile ranges within 14%. The model is calibrated on $P = 1$.

Convergence of CFR, CFR$^+$ and DCFR



Exploitability in percent of the pot; the dashed line is the 0.3% target. The fitted slopes of $\log \text{expl}$ against $\log T$ at the end of each run are about -1 for CFR, -1.4 to -1.7 for CFR$^+$ and -1.4 to -2.1 for DCFR.

Iterations to a fixed accuracy

DCFR on five river, five turn and three flop spots, with one tree per street: median and range of the iterations needed for each accuracy, in percent of the pot. Counts are the first checkpoint below the target, and checkpoints are 15% apart.

street	spots	1%	0.3%	0.1%
river	5	46 (46–53)	95 (82–110)	170 (147–226)
turn	5	95 (82–95)	170 (147–196)	299 (299–344)
flop	3	40 (34–40)	71 (61–71)	110 (110–127)

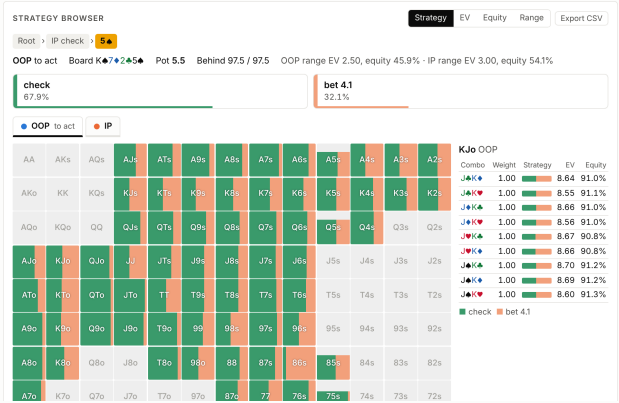
The iterations vary by less than a factor of two between spots. On one flop (right) they depend on the betting structure, not on the number of nodes: both trees without river betting need 40, one size on every street needs 71, and a flop check-raise raises this to 110. The running time is governed by the cost per iteration.

K72 tree	decisions	0.3%	0.1%
later streets checked down	8	40	71
river checked down	789	40	71
one size per street	56,842	71	110
standard	85,263	110	196

Validation

test	reference	result
River, talk example	the earlier implementation	same iterates: exploitability 1.34×10^{-4} and 1.34×10^{-4} chips after 5000 iterations
Turn, one chance level	exact LP, 14 and 12 hands	value 24.92419862 against 24.92419856
Flop, two chance levels	exact LP, 6 and 5 hands	value 27.139343 against 27.139333
Turn, mid size	exact LP, 40 and 35 hands	value 2.26976132 against 2.26976132
Heads-up push or fold	LP on the same tables	values agree to five digits
Parallel partition	sequential solver	values and exploitability agree to 10^{-14} relative
Zero-reach shortcut	traversal without shortcut	regrets and sums bit for bit
Leaf evaluation	direct $O(n^2)$ sums	agreement to 10^{-12}

Demonstration



Strategy browser after a flop solve: the big blind's turn strategy after the flop checks through.

The site has five pages: postflop, preflop and multiway, flop reports, the river table of the earlier talk and a methods page. In the demonstration we

1. solve a flop and follow the exploitability,
2. browse strategies, EV and equity by turn card,
3. lock a node and re-solve,
4. solve a push-or-fold game and a six-player game.

Node locking

A locked node keeps a fixed strategy $\hat{\sigma}$: its regrets are not updated, the traversals use $\hat{\sigma}$, and best responses do not optimize over it. DCFR then solves the **restricted game**

$$\min_{y \in Y} \max_{x \in X,\ x|_{\text{locked}} = \hat{\sigma}} x^\top A y.$$

Example on the river: after a check the button is locked to bet a third of the pot with every hand. In equilibrium it never makes that bet, so the big blind's response to it is off the equilibrium path and not determined by the game.

	equilibrium	against the lock
button bets a third of the pot after a check	0%	100%
big blind folds to that bet	49%	29%
big blind value per hand, chips	-16.2	-6.2
iterations to 0.3% of the pot	40	27

Aggregate flop reports

The symmetric group S_4 acts on the suits. Its orbits on the $\binom{52}{3} = 22{,}100$ flops are the **1755** strategically different flops, with orbit sizes 4, 12 and 24.

A report solves one spot on a subset and averages with weights equal to the orbit sizes represented. Subsets are stratified by texture and by rank, so the average is a stratified quadrature over all flops. Standard errors come from pairing neighbouring strata. Each flop is solved to 0.05% of the pot, which bounds its value error by twice that.

flops	big blind value	solver bound	big blind bets
25	-0.450 ± 0.018	0.0047	19.8 ± 0.9%
49	-0.436 ± 0.019	0.0048	21.0 ± 1.0%
95	-0.453 ± 0.012	0.0048	20.9 ± 0.6%

Weighted averages, standard errors and the solver's error bound on the flop tree with checked-down turn and river. Values are in chips with a pot of 5.5.

Preflop: card removal between hand classes

Preflop strategies are per class, 169 of them. A hand that sees a flop is settled by all-in equity, so postflop play is not modelled. Card removal between classes i and j is the exact fraction of compatible pairs

$$M_{ij} = \frac{\#\{\text{compatible pairs of combinations}\}}{|i|\,|j|}, \qquad M = \mathbf{1}\mathbf{1}^{\top} - D,$$

where D is nonzero only for classes that share a rank (27% of the entries). With two players this card-removal model is exact.

stack	LP value	DCFR value	push	call
5 bb	0.05616	0.05616	71.5%	62.0%
10 bb	-0.04538	-0.04538	58.2%	37.3%
15 bb	-0.12721	-0.12721	45.2%	28.5%
20 bb	-0.18316	-0.18316	40.2%	21.8%

Heads-up push or fold: DCFR against a linear program on the same tables.

Payoff errors and exploitability

Lemma

If the payoffs of a game are perturbed by at most δ at every terminal node, then every expected payoff changes by at most δ, and an ε-equilibrium of the perturbed game is an $(\varepsilon + \delta)$-equilibrium of the original game.

The equity tables are Monte Carlo estimates. Two independent tables differ by at most 0.012, so with δ the pot times that difference each solution is an $(\varepsilon + \delta)$-equilibrium of the other game. The measured cross-exploitability is four to five orders of magnitude smaller:

stack	value	value change	cross-exploitability	δ
5 bb	0.05616	3.0×10^{-5}	1.4×10^{-5}	0.12
10 bb	-0.04538	8.0×10^{-5}	2.8×10^{-6}	0.23
15 bb	-0.12721	2.8×10^{-5}	3.4×10^{-5}	0.35
20 bb	-0.18316	1.6×10^{-4}	5.8×10^{-6}	0.47

Three-way showdowns: a low-rank factorization

A three-way showdown needs $\sum_{j,k} a_j b_k F_i(j,k)$ for every i, where $F_i(j,k)$ is the equity of i against j and k: 169^3 operations per leaf. A symmetric eigendecomposition truncated to rank R,

$$F_i \approx \sum_{r=1}^{R} \lambda_{ir}\, u_{ir} u_{ir}^{\top},$$

reduces this to $2 \cdot 169 \cdot R$ per class.

R	slice error	contraction error
1	0.19	0.016
4	0.14	6.3×10^{-3}
8	0.10	4.8×10^{-3}
16	0.059	2.7×10^{-3}
32	0.047	2.5×10^{-3}

The slices are not close to low rank, but the contraction with nonnegative ranges is: rank 16 gives a relative error of 2.7×10^{-3} at about a fifth of the cost, since $169^2/(2 \cdot 169 \cdot 16) \approx 5.3$.

Multiway convergence

With three or more players the game is not zero-sum between two sides, and DCFR has no convergence guarantee. We measure NashConv, the sum over players of the gain from a best response.

Push-or-fold games reach a NashConv of about 10^{-6} bb. The 20 bb six-player tree stalls near 0.06 bb per hand; the 100 bb tree is still decreasing at the end of the run.

game	nodes	ms/it.	iter.	NashConv
3 players, push/fold	6	4	843	4.6×10^{-7}
6 players, push/fold	55	69	843	1.4×10^{-6}
6 players, 20 bb	1,197	65	843	0.061
6 players, 100 bb	6,842	289	843	0.043

NashConv in big blinds per hand after the stated number of iterations; milliseconds per iteration on one core.

Limitations

- The flop takes seconds per iteration in a browser and grows with every added bet size. Commercial solvers are native code with vector instructions and suit isomorphism.
- The proven rate of DCFR is $O(T^{-1/2})$ with a vacuous constant. The fast observed rates are not explained by theory.
- Preflop play treats postflop as an all-in showdown, so it overvalues hands that realize equity poorly.
- Multiway solutions have no equilibrium guarantee and use an independence approximation for card removal between opponents.
- Parallel scaling has been measured on two cores only.
- The flop solver has not yet been compared with an established solver on the same tree; the exact checks cover games of up to 40 hands per player.

Next steps

- The first step is a comparison of values and exploitability with an established open-source solver on the same flop trees.
- A high-precision finish restricts the game to the DCFR support and solves the equilibrium conditions with a proximal semismooth Newton method, which reached 10^{-13} on river subgames in our earlier work.
- Suit isomorphism solves one representative of each class of isomorphic turn and river cards and saves up to a factor of four on monotone flops.
- Compiled kernels in WebAssembly with SIMD can speed up the vector loops, and GPU kernels the leaf sweeps.
- Regret-based pruning can skip subtrees with persistently negative regret, with a correction that keeps the iterates exact.
- A preflop solver with realization would replace the all-in showdown by values from postflop solves.
- A theory of the observed DCFR rates on poker trees is open.

Conclusions

- One method, discounted CFR, solves flop, turn and river subgames, locked games and preflop games.
- Range vectorization makes each iteration a sequence of vector operations, and inclusion–exclusion with sorted sweeps makes every leaf linear in the number of hands.
- A two-term cost model accounts for the time per iteration, single precision storage reaches the same exploitability, and the partition by chance card reproduces the sequential results.
- DCFR reaches practical accuracy in a few hundred iterations on every street we measured, so the cost per iteration decides the running time.

Site, source and experiments: `github.com/dwangnumbers/Poker-Solver`

References

- D. Koller, N. Megiddo, B. von Stengel. Efficient computation of equilibria for extensive two-person games. *Games Econ. Behav.* 14 (1996).
- S. Hart, A. Mas-Colell. A simple adaptive procedure leading to correlated equilibrium. *Econometrica* 68 (2000).
- M. Zinkevich, M. Johanson, M. Bowling, C. Piccione. Regret minimization in games with incomplete information. *NeurIPS* 2007.
- M. Johanson, K. Waugh, M. Bowling, M. Zinkevich. Accelerating best response calculation in large extensive games. *IJCAI* 2011.
- O. Tammelin. Solving large imperfect information games using CFR$^+$. arXiv:1407.5042 (2014).
- N. Brown, T. Sandholm. Solving imperfect-information games via discounted regret minimization. *AAAI* 2019.
- N. Brown, T. Sandholm. Superhuman AI for multiplayer poker. *Science* 365 (2019).
- N. Burch, M. Moravčík, M. Schmid. Revisiting CFR$^+$ and alternating updates. *J. Artif. Intell. Res.* 64 (2019).

Appendix: the showdown sweep

The input is the own list sorted by strength q_h, the opponent list sorted by strength and the opponent reach r.

1. Set $\tau = 0$ and $s_c = 0$ for all cards c.
2. For each block of own hands with equal strength q:
   2.1 add every opponent hand j with $q_j < q$ to τ and to s_c for its two cards;
   2.2 for each own hand h of the block set $L(h) = \tau - s_{h_1} - s_{h_2}$;
   2.3 add every opponent hand with $q_j = q$, and set $L^{=}(h) = \tau - s_{h_1} - s_{h_2} + r_{\bar{h}}$.
3. After the sweep, $m(h)$ is the fold formula with the final sums.

Each opponent hand is added once and each own hand is read twice, so the cost is $O(n_1 + n_2)$.

Appendix: counting flops with Burnside's lemma

The number of orbits of S_4 on flops is the average number of flops fixed by a suit permutation g:

$$\#\text{orbits} = \frac{1}{24}\sum_{g \in S_4} |\mathrm{Fix}(g)| = \frac{22100 + 6 \cdot 2938 + 8 \cdot 299}{24} = 1755.$$

cycle type	count	flops fixed
identity	1	all 22,100
transposition $(a\,b)$	6	$\binom{26}{3}$ in the other two suits, plus a pair of equal rank in a and b with one more card: $2600 + 338$
3-cycle	8	$\binom{13}{3}$ in the fixed suit, plus one rank in the three cycled suits: $286 + 13$
double transposition, 4-cycle	3, 6	none

The code maps each flop to its smallest image under the 24 permutations and finds the same 1755 classes.

Appendix: the earlier high-precision results

The previous talk solved river subgames to machine precision with a proximal semismooth Newton method started from DCFR, with interior-point crossover and support restriction.

game	DCFR	DCFR then Newton
Leduc hold'em	1.0×10^{-5} in 14 s	5.5×10^{-14} in 0.33 s
small river	5.4×10^{-6} in 9 s	3.9×10^{-14} in 1.9 s
river of the talk	1.3×10^{-4} in 16 s	3.2×10^{-5} in 528 s

Exploitability in chips with a pot of 100. On the largest river the Newton phase was slower than DCFR, which is why the method is listed as a next step and not as part of the solver.